

A LIPSCHITZ INTEGER GRAPH WITHOUT THREE-TERM ARITHMETIC PROGRESSIONS

JENSEN KOHLMAYER AND LIAM KRUER

ABSTRACT. We construct a strictly increasing function $F: \mathbb{N} \rightarrow \mathbb{Z}$ whose graph contains no nontrivial three-term arithmetic progression and whose successive differences lie in a specified set of sixteen positive integers, all at most 125. Thus F is 125-Lipschitz, giving an affirmative answer to Problem 15 in Green’s collection. Equivalently, its difference sequence is an infinite word over a finite set of positive integers with no additive square.

The construction starts with a four-uniform substitution on six letters. A bounded correction depending on the endpoint letter converts an equal-sum condition into equality of three endpoint letters. The remaining obstruction is excluded by an invariant with 3 688 states. Division of the three indices by four preserves the required error bounds, so strong induction extends the finite verification to all indices. We give the complete construction, a short reproducible certificate computation, and a Lean formalization whose finite checks are reduced in the kernel. The formal proof uses only the axioms permitted by the pinned Green 15 challenge.

1. INTRODUCTION

We write $\mathbb{N} = \{0, 1, 2, \dots\}$. Green’s Problem 15 asks whether there is a Lipschitz function $F: \mathbb{N} \rightarrow \mathbb{Z}$ for which

$$\Gamma_F = \{(n, F(n)) : n \in \mathbb{N}\} \subseteq \mathbb{Z}^2$$

contains no nontrivial three-term arithmetic progression [4, Problem 15]. For a graph of this form, the required condition is

$$i + k = 2j, \quad F(i) + F(k) = 2F(j) \implies i = j = k. \quad (1.1)$$

If F is strictly increasing, this is also the absence of a *double* three-term progression in the sequence of its values. Brown, Jungić and Poelstra [1] discuss this formulation and its relation to the additive-square question associated with Brown and Freedman.

For comparison, Cassaigne, Currie, Schaeffer and Shallit [2] constructed an infinite word over $\{0, 1, 3, 4\}$ with no three consecutive blocks of the same length and sum. Taking partial sums gives a Lipschitz graph without four-term progressions. The construction below excludes three-term progressions.

Theorem 1.1. *There is an explicitly defined function $F: \mathbb{N} \rightarrow \mathbb{Z}$ such that $F(0) = 0$, F is strictly increasing, and*

$$\begin{aligned} F(n+1) - F(n) &\in \mathcal{D} \quad (n \in \mathbb{N}), \\ \mathcal{D} &= \{1, 3, 4, 9, 20, 30, 39, 49, 80, 90, 96, 106, 117, 119, 120, 125\}. \end{aligned} \quad (1.2)$$

Moreover, (1.1) holds. In particular, $|F(n) - F(m)| \leq 125|n - m|$ for all $m, n \in \mathbb{N}$.

Date: September 10, 2026.

2020 Mathematics Subject Classification. Primary 11B25; Secondary 68R15, 68V20.

Key words and phrases. arithmetic progressions, additive squares, substitutions, finite-state certificates, Lean formalization.

An *additive square* in an integer word $v_0v_1v_2\cdots$ consists of two consecutive nonempty blocks of the same length and the same sum. Consequently, Theorem 1.1 has the following equivalent consequence.

Corollary 1.2. *There is an infinite additive-square-free word over $\mathcal{D}$.*

Proof. Put $v_n = F(n+1) - F(n)$. Equality of the sums on $[i, i+\ell]$ and $[i+\ell, i+2\ell]$, with $\ell \geq 1$, is precisely $F(i) + F(i+2\ell) = 2F(i+\ell)$, contrary to (1.1). $\square$

The proof separates the arithmetic encoding from the finite computation. The encoding forces equal endpoint letters whenever the graph has a three-term progression. A bounded-state invariant then excludes that configuration. Its closure, together with an elementary descent argument, is the part of the proof that covers arbitrarily large indices.

2. THE SUBSTITUTION AND THE ENDPOINT ENCODING

Let $\mathcal{A} = \{a, b, c, d, e, f\}$. The substitution $h: \mathcal{A} \rightarrow \mathcal{A}^4$, weight $w: \mathcal{A} \rightarrow \mathbb{Z}$, and tag $q: \mathcal{A} \rightarrow \mathbb{Z}$ are given by Table 1. Positions inside substitution images are numbered 0, 1, 2, 3.

TABLE 1. The six-letter construction.

α	$h(\alpha)$	$w(\alpha)$	$q(\alpha)$
a	$acab$	0	0
b	$aeab$	1	1
c	$acfb$	2	3
d	$fead$	4	4
e	$fcfd$	5	9
f	$fefd$	6	10

Since $h(f)$ starts with f , the words $h^t(f)$ are nested prefixes and define an infinite fixed point $u = h^\omega(f)$. In particular,

$$u_0 = f, \quad u_{4n+r} = h(u_n)_r \quad (n \in \mathbb{N}, 0 \leq r < 4). \quad (2.1)$$

Define

$$S(n) = \sum_{t=0}^{n-1} w(u_t), \quad G(n) = S(n) - 3n, \quad F(n) = 21S(n) + q(u_n) - 10. \quad (2.2)$$

The empty sum is zero and $q(f) = 10$, so $F(0) = 0$.

Lemma 2.1. *For $\alpha, \beta, \gamma \in \mathcal{A}$,*

$$q(\alpha) - 2q(\beta) + q(\gamma) = 0 \implies \alpha = \beta = \gamma.$$

Proof. The six distinct tag values 0, 1, 3, 4, 9, 10 have only digits 0 and 1 in base three. Neither addition of two such numbers nor multiplication of one by two produces a carry. Equality $q(\alpha) + q(\gamma) = 2q(\beta)$ therefore holds digit by digit. Each digit of the middle term forces the other two digits to agree with it. The tag values, and hence the letters, are equal. $\square$

The following observation explains the coefficient 21 in (2.2).

Lemma 2.2 (Endpoint separation). *If $i, j, k \in \mathbb{N}$ satisfy $F(i) - 2F(j) + F(k) = 0$, then*

$$S(i) - 2S(j) + S(k) = 0, \quad u_i = u_j = u_k.$$

Proof. The assumed equality becomes

$$21(S(i) - 2S(j) + S(k)) + q(u_i) - 2q(u_j) + q(u_k) = 0.$$

The tag contribution is between -20 and 20 . It is a multiple of 21 , so it is zero. The weight-sum contribution is zero as well, and Lemma 2.1 identifies the three letters. $\square$

In terms of increments, the tag adds the bounded term $q(u_{n+1}) - q(u_n)$. Its sum over a block depends only on the two endpoint letters. More generally, the same argument works with a tag in $[0, Q]$ whose image is progression-free and a multiplier greater than $2Q$.

Proposition 2.3. *The function in (2.2) satisfies (1.2), is strictly increasing, and is 125-Lipschitz.*

Proof. For every n ,

$$F(n+1) - F(n) = 21w(u_n) + q(u_{n+1}) - q(u_n). \quad (2.3)$$

In each image of h , positions $0, 2$ belong to $\{a, f\}$ and positions $1, 3$ belong to $\{b, c, d, e\}$. Hence every adjacent pair in u belongs to

$$\{ab, ac, ad, ae, ba, bf, ca, cf, da, df, ea, ef, fb, fc, fd, fe\}. \quad (2.4)$$

Evaluating (2.3) on these pairs gives, in the displayed order, the sixteen elements of $\mathcal{D}$ in (1.2). They lie between 1 and 125 , proving strict increase. Summing the increment bound from m to $n-1$ when $m < n$ gives the Lipschitz estimate. $\square$

3. A FINITE INVARIANT FOR TRIPLES OF INDICES

3.1. The centered recurrence. For $\alpha \in \mathcal{A}$ and $0 \leq r < 4$, put

$$P(\alpha, r) = \sum_{s=0}^{r-1} w(h(\alpha)_s), \quad B(\alpha, r) = P(\alpha, r) - 3r.$$

Direct inspection of Table 1 gives

$$\sum_{s=0}^3 w(h(\alpha)_s) = 3w(\alpha) + 3. \quad (3.1)$$

The complete table of B is as follows.

TABLE 2. The correction $B(\alpha, r)$.

$\alpha \backslash r$	0	1	2	3
a	0	-3	-4	-7
b	0	-3	-1	-4
c	0	-3	-4	-1
d	0	3	5	2
e	0	3	2	5
f	0	3	5	8

Lemma 3.1. *For $n \in \mathbb{N}$ and $0 \leq r < 4$,*

$$S(4n+r) = 3S(n) + 3n + P(u_n, r), \quad (3.2)$$

$$G(4n+r) = 3G(n) + B(u_n, r). \quad (3.3)$$

Proof. The first $4n$ letters of u are the images of its first n letters. Summing (3.1) over these letters, and then adding the first r weights in $h(u_n)$, proves (3.2). Subtracting $3(4n+r)$ proves (3.3). $\square$

3.2. States and transitions. For a triple $i, j, k \in \mathbb{N}$, define its errors

$$A(i, j, k) = i - 2j + k, \quad D(i, j, k) = G(i) - 2G(j) + G(k), \quad (3.4)$$

and let $\eta(i, j, k)$ be true exactly when the three indices are not all equal. Its state is

$$\sigma(i, j, k) = (u_i, u_j, u_k, A(i, j, k), D(i, j, k), \eta(i, j, k)).$$

The finite state space used in the proof is

$$\mathcal{Q} = \mathcal{A}^3 \times \{-1, 0, 1\} \times \{-15, -14, \dots, 15\} \times \{\text{false}, \text{true}\}. \quad (3.5)$$

Thus $|\mathcal{Q}| = 6^3 \cdot 3 \cdot 31 \cdot 2 = 40\,176$.

For digits $x, y, z \in \{0, 1, 2, 3\}$, the transition from $(\alpha, \beta, \gamma, A, D, \eta)$ is

$$\begin{aligned} \alpha' &= h(\alpha)_x, & \beta' &= h(\beta)_y, & \gamma' &= h(\gamma)_z, \\ A' &= 4A + x - 2y + z, \\ D' &= 3D + B(\alpha, x) - 2B(\beta, y) + B(\gamma, z), \\ \eta' &= \eta \text{ or } (x \neq y \text{ or } y \neq z). \end{aligned} \quad (3.6)$$

We retain a transition exactly when its new errors satisfy $|A'| \leq 1$ and $|D'| \leq 15$.

These formulas describe the exact passage from $\sigma(I, J, K)$ to $\sigma(4I+x, 4J+y, 4K+z)$. The letter and height formulas follow from (2.1) and (3.3). The Boolean formula follows from uniqueness of quotient and remainder on division by four.

Lemma 3.2 (Descent bounds). *If $\sigma(i, j, k) \in \mathcal{Q}$, and $I = \lfloor i/4 \rfloor$, $J = \lfloor j/4 \rfloor$, $K = \lfloor k/4 \rfloor$, then $\sigma(I, J, K) \in \mathcal{Q}$.*

Proof. Write $i = 4I + x$, $j = 4J + y$, and $k = 4K + z$, with digits between 0 and 3. The index correction has absolute value at most 6. Therefore

$$|A(I, J, K)| \leq \frac{|A(i, j, k)| + 6}{4} \leq \frac{7}{4} < 2,$$

so the integral error lies in $\{-1, 0, 1\}$. Table 2 places every value of B in $[-7, 8]$, whence

$$|B(\alpha, x) - 2B(\beta, y) + B(\gamma, z)| \leq 30.$$

The height recurrence consequently gives

$$|D(I, J, K)| \leq \frac{|D(i, j, k)| + 30}{3} \leq 15.$$

These are exactly the required bounds. $\square$

3.3. The certificate. Let $s_0 = (f, f, f, 0, 0, \text{false})$. Define $\mathcal{R}$ to be the smallest subset of $\mathcal{Q}$ that contains s_0 and is closed under every retained transition (3.6). It can be computed by starting with $\{s_0\}$ and repeatedly adjoining the retained successors of every newly adjoined state. This procedure terminates after processing at most 40 176 states.

Proposition 3.3 (Finite certificate). *The set $\mathcal{R}$ has 3 688 states and 19 020 retained outgoing transitions, counted with their digit triples. It contains no state of the form*

$$(\alpha, \alpha, \alpha, 0, 0, \text{true}), \quad \alpha \in \mathcal{A}. \quad (3.7)$$

Finite verification. Appendix B gives a complete exact-integer computation from Tables 1 and 2. It exhausts the retained successors of every state, checks the two counts, and excludes (3.7). The Lean file includes the resulting invariant as integer bit masks and checks its initial state, closure, and exclusion of (3.7) by kernel reduction; see Section 5. Only these three properties, rather than the counts or minimality of the invariant, are used below. $\square$

Lemma 3.4. *If $\sigma(i, j, k) \in \mathcal{Q}$, then $\sigma(i, j, k) \in \mathcal{R}$.*

Proof. Use strong induction on $i + j + k$. For sum zero, all three indices vanish and the state is s_0 . Otherwise form the quotient triple (I, J, K) and the remainders (x, y, z) as in Lemma 3.2. That lemma puts the quotient state in $\mathcal{Q}$, and $I + J + K < i + j + k$. The induction hypothesis therefore puts it in $\mathcal{R}$. Applying (3.6) with (x, y, z) recovers $\sigma(i, j, k)$. The transition is retained because the latter state belongs to $\mathcal{Q}$, so closure proves the assertion. $\square$

Corollary 3.5 (Boundary obstruction). *If $i, j, k \in \mathbb{N}$ satisfy*

$$i - 2j + k = 0, \quad G(i) - 2G(j) + G(k) = 0, \quad u_i = u_j = u_k,$$

then $i = j = k$.

Proof. Both errors vanish, so Lemma 3.4 places the state in $\mathcal{R}$. If the indices were not all equal, this would be a state of the forbidden form (3.7). $\square$

4. PROOF OF THE MAIN THEOREM

Proof of Theorem 1.1. Use the function F in (2.2). Its value at zero and its increment, monotonicity, and Lipschitz properties were established in Section 2. Suppose now that

$$i - 2j + k = 0, \quad F(i) - 2F(j) + F(k) = 0.$$

Lemma 2.2 gives $S(i) - 2S(j) + S(k) = 0$ and $u_i = u_j = u_k$. Since $G(n) = S(n) - 3n$,

$$G(i) - 2G(j) + G(k) = S(i) - 2S(j) + S(k) - 3(i - 2j + k) = 0.$$

Corollary 3.5 now yields $i = j = k$, as required. $\square$

The difference word of Corollary 1.2 can also be generated directly by a substitution on its sixteen integer symbols; the complete table appears in Appendix A. The value 125 is an explicit upper bound for this construction. No assertion that it is the smallest possible Lipschitz constant is needed for the theorem.

5. LEAN FORMALIZATION AND REPRODUCIBILITY

The ancillary file `Green15.lean` contains the complete formal proof, including the finite invariant. The declarations are placed in namespace `Bounty` by the challenge's trusted wrapper. The theorem `Bounty.main_result` states all the properties in Theorem 1.1: the value at zero, strict monotonicity, membership of every increment in $\mathcal{D}$, the Lipschitz constant 125, and the progression-free graph. The final theorem is the exact challenge target:

```
theorem target : fcTypeOfName% "Green15.green_15" := by
  exact Iff.intro
    (fun _ => Exists.intro 125
      (Exists.intro f (And.intro f_lipschitz f_graph_free)))
    (fun _ => True.intro)
```

The elaborated target is

$$\text{True} \iff \exists K \in \mathbb{R}_{\geq 0} \exists F: \mathbb{N} \rightarrow \mathbb{Z}, \quad \text{LipschitzWith}(K, F) \wedge \text{APFree}_3(\Gamma_F).$$

5.1. The infinite construction. The formal functions `letter` and `height` are defined by well-founded recursion on n , using $\lfloor n/4 \rfloor$ when $n > 0$. They satisfy (2.1) and (3.3), with initial values f and 0. The theorem `height_succ` proves

$$G(n+1) - G(n) = w(u_n) - 3.$$

Together with $G(0) = 0$, this identifies the recursive height with $S(n) - 3n$ by induction. The formal function `f` is $21(G(n) + 3n) + q(u_n) - 10$, hence is precisely (2.2).

5.2. The finite computation. Letters are represented by $0, 1, \dots, 5$ in the order $a, b, \dots, f$. For each letter triple (α, β, γ) and $A \in \{-1, 0, 1\}$, the file stores a natural-number mask in row

$$(36\alpha + 6\beta + \gamma) 3 + (A + 1).$$

For a state with Boolean flag true, bit $D + 15$ of that mask specifies membership. The flag-false branch consists of equal-letter states with $A = D = 0$. Thus the table has 648 rows and records a subset of (3.5) without any external data file.

The six theorems `checkA110`, ..., `checkA115` check all retained transitions, grouped by the first letter. They use `decide +kernel`. A separate theorem proves that the Boolean checker implies the quantified closure property. The theorem `invariant_descent` is the strong-induction proof of Lemma 3.4. The passage from the midpoint criterion (1.1) to the library definition `Set.IsAPOfLengthFree` is also formalized.

Both `Bounty.main_result` and `Bounty.target` have exactly the reported transitive axiom dependencies

$$\text{propext}, \quad \text{Classical.choice}, \quad \text{Quot.sound}.$$

The development contains no unproved placeholder and does not use `native_decide`. In particular, the finite assertion is not imported as an axiom or justified by a floating-point computation.

5.3. Files and checks. The development was checked with Lean 4.33.1 and the Formal Conjectures revision [3]

$$8432\text{eac}998110\text{a}563\text{e}03\text{df}65\text{a}28\text{c}117\text{e}97\text{c}8\text{c}142.$$

The ancillary validation record gives the source and target type hashes, the Lean-file hash, and the axiom audit. It also records that the entire upload file passes the submission-policy scanner. The compilation uses the exact trusted `SolutionHeader.lean.txt` and `SolutionFooter.lean.txt`, with their hashes checked against the task manifest. The upload file itself therefore has no import commands and no outer namespace declaration; these are supplied by the wrapper. Local reconstruction instructions are included with the source files.

The short checker in Appendix B independently regenerates the invariant using ordinary Python integers. Longer prefix searches were used during development, but no finite-prefix bound enters the proof of Theorem 1.1. The all-indices statement follows from the descent lemma and the checked invariant.

APPENDIX A. A SUBSTITUTION ON THE SIXTEEN INCREMENTS

For an allowed adjacent pair, write $L(\alpha, \beta) = 21w(\alpha) + q(\beta) - q(\alpha)$. The sixteen pairs in (2.4) have distinct labels, namely $\mathcal{D}$. Replacing a pair (α, β) by the four consecutive pairs in the five-letter word

$$h(\alpha)_0 h(\alpha)_1 h(\alpha)_2 h(\alpha)_3 h(\beta)_0$$

and then applying L yields Table 3.

The word u begins with fe , whose pair label is 125. Its adjacent-pair sequence is fixed by the induced pair substitution, so the increment word is exactly $v = \theta^\omega(125)$. In particular, an entirely numerical description of the function is

$$F(n) = \sum_{t=0}^{n-1} v_t.$$

Table 3 is an alternative presentation of the same construction; the proof of avoidance uses the six-letter representation.

TABLE 3. The substitution θ on $\mathcal{D}$.

d	$\theta(d)$	d	$\theta(d)$
1	(3, 39, 1, 20)	80	(125, 96, 4, 80)
3	(3, 39, 1, 20)	90	(125, 96, 4, 90)
4	(3, 39, 1, 30)	96	(119, 49, 120, 80)
9	(3, 39, 1, 30)	106	(119, 49, 120, 90)
20	(9, 96, 1, 20)	117	(125, 106, 120, 80)
30	(9, 96, 1, 30)	119	(125, 106, 120, 80)
39	(3, 49, 117, 20)	120	(125, 106, 120, 90)
49	(3, 49, 117, 30)	125	(125, 106, 120, 90)

APPENDIX B. A COMPLETE FINITE-STATE CHECKER

The following Python 3 code has no nonstandard dependencies. It contains all the data needed for Proposition 3.3. The six letters are encoded as 0, . . . , 5; the last coordinate of a state is a Boolean. Every transition kept by the stated bounds is examined.

```

from itertools import product

H = ((0,2,0,1), (0,4,0,1), (0,2,5,1),
      (5,4,0,3), (5,2,5,3), (5,4,5,3))
B = ((0,-3,-4,-7), (0,-3,-1,-4), (0,-3,-4,-1),
      (0,3,5,2), (0,3,2,5), (0,3,5,8))
digits = tuple(product(range(4), repeat=3))
root = (5,5,5,0,0,False)
seen, pending, edges = {root}, [root], 0

while pending:
    a,b,c,A,D,e = pending.pop()
    if A == D == 0 and e and a == b == c:
        raise RuntimeError("forbidden state")
    for x,y,z in digits:
        AA = 4*A + x - 2*y + z
        DD = 3*D + B[a][x] - 2*B[b][y] + B[c][z]
        if abs(AA) > 1 or abs(DD) > 15:
            continue
        t = (H[a][x], H[b][y], H[c][z], AA, DD,
              e or x != y or y != z)
        edges += 1
        if t not in seen:
            seen.add(t)
            pending.append(t)

if (len(seen), edges) != (3688, 19020):
    raise RuntimeError("unexpected certificate counts")
print(f"{len(seen)} states; {edges} retained transitions")

```

The output is 3688 states; 19020 retained transitions. The raise statements are actual checks, so the exclusion does not depend on whether Python assertions are enabled. This executable computation is supplied for reproduction; the accompanying Lean certificate supplies kernel-checked closure and exclusion.

Acknowledgments and AI assistance. OpenAI Codex was used in the computational search that produced the substitution, the development of the endpoint encoding and finite-state argument, the verification programs, the Lean formalization, and the preparation of this manuscript. We credit this substantial AI assistance explicitly. The named authors are responsible for the final manuscript. The mathematical certification reported here is the Lean kernel check described in Section 5.

REFERENCES

- [1] T. Brown, V. Jungić and A. Poelstra, *On double 3-term arithmetic progressions*, Integers **14** (2014), Paper A43. <https://math.colgate.edu/~integers/o43/o43.pdf>.
- [2] J. Cassaigne, J. D. Currie, L. Schaeffer and J. Shallit, *Avoiding three consecutive blocks of the same size and same sum*, J. ACM **61** (2014), no. 2, Article 10. <https://arxiv.org/abs/1106.5204>.
- [3] The Formal Conjectures Authors, *Formal Conjectures*, file FormalConjectures/GreensOpenProblems/15.lean, revision 8432eac998110a563e03df65a28c117e97c8c142. <https://github.com/google-deepmind/formal-conjectures>.
- [4] B. Green, *100 open problems*, 2024, Problem 15. <https://people.maths.ox.ac.uk/greenbj/papers/open-problems.pdf>.

Email address: jenwin@purdue.edu

Email address: kruerl@purdue.edu